

01. What happens when you call a function in JavaScript? **(2Marks)**

- (A) The function is deleted
- (B) The function's code runs
- (C) The function is created
- (D) Nothing happens

02. What is object destructuring in JavaScript? **(2Marks)**

- (A) A way to delete properties from an object
- (B) A way to copy an entire object into another object
- (C) A way to extract properties from an object into separate variables
- (D) A way to merge two objects together

03. Answer by True or False **(2Marks)**

REST stands for Representational State Transfer, and it is a set of rules for building web services that communicate over HTTP.

04. Answer by True or False **(2Marks)**

The HTTP GET method is used to send new data to the server to create a new record.

05. Daniel is building a registration form with three fields: name, email, and age. He needs to store each field's value separately. **(2Marks)**

Which of the following statements about `useState` in React is correct?

- (A) `useState` can only be declared once per component
- (B) `useState` can be declared a maximum of three times per component
- (C) All form data must be stored in a single `useState` declaration
- (D) `useState` can be declared multiple times to manage different pieces of data

06. Answer by True or False **(2Marks)**

When rendering a list in React, each item must have a unique key prop so React can efficiently track which items changed, were added, or were removed.

07. **(2Marks)**

Linda is selecting elements on her web page using JavaScript. Her teacher explains that different DOM methods work in different ways.

Explain what makes `document.getElementById()` different from `document.getElementsByClassName()`. Which part of an HTML element does each one use to find elements?

- (A) `getElementById` uses the tag name; `getElementsByClassName` uses the id attribute
- (B) `getElementById` uses the unique id attribute; `getElementsByClassName` uses the class attribute shared by multiple elements
- (C) Both methods work the same way but `getElementById` is faster
- (D) `getElementById` selects all matching elements; `getElementsByClassName` selects only the first match

08. Maria wants to remove a child element from the page using JavaScript DOM. **(2Marks)**

`parentElement. _____ (childElement);`

- (A) `deleteChild`
- (B) `eraseChild`
- (C) `removeChild`
- (D) `destroyChild`

09. **(2Marks)**

Niyonzima sees the following Express middleware and wonders what `next()` does:

What happens if `next()` is not called?

```
app.use((req, res, next) => {  
  console.log("Request received");  
  next();  
});
```

- (A) The server restarts automatically
- (B) The request moves to the next middleware anyway
- (C) The request will hang and never reach the next middleware or route
- (D) An error is thrown immediately

10. Answer by True or False (2Marks)

Niyonsaba wraps her entire app with the Redux <Provider> component. His teammate asks why.

She replied, “The <Provider> component is used to make the Redux store available to all components in the application.”

11. Ingabire has a list of product prices and wants to display only the prices that are greater than (3Marks)

500 RWF. The code structure is ready. She just needs the loop and condition.

Write the missing code to loop through the list and display only prices greater than 500.

```
let output = document.getElementById("results");
let prices = [200, 750, 300, 1200, 450, 900];
let text = "";

// ===== WRITE YOUR CODE HERE =====

// ===== STOP WRITING HERE =====

output.innerHTML = text;
```

Expected Output: "750 1200 900"

12. Habimana is creating a Course model for his school API. Each course must have a **name** (3Marks)

(String, required, minimum 3 characters) and a **price** (Number, required, minimum 0). The connection to MongoDB is already done. He just needs to define the schema and create the model.

```
const mongoose = require("mongoose");
mongoose.connect("mongodb://localhost/schoolDB");

// ===== WRITE YOUR CODE HERE =====

// ===== STOP WRITING HERE =====

module.exports = Course;
```

Expected behavior: A Course model is created with name and price fields, both required, with minimum value rules.

What you need to do:

- Create a new mongoose.Schema with the validation rules described above
- Create a model called Course from the schema
- You need to write **6 to 8 lines** of code

13. Mutoni has a list of students in an array. She needs a **DELETE endpoint** that removes a student by their ID. The Express app and the array are ready. She just needs the delete route.

(3Marks)

```
const express = require("express");
const app = express();

const students = [
  { id: 1, name: "Uwase" },
  { id: 2, name: "Kagabo" },
  { id: 3, name: "Ishimwe" }
];

// ===== WRITE YOUR CODE HERE =====

// ===== STOP WRITING HERE =====

app.listen(3000);
```

Expected behavior: When Postman sends DELETE /api/students/2, the student with id 2 is removed and the deleted student is sent back as the response.

What you need to do:

- Create a DELETE route at /api/students/:id
- Find the student that matches the id
- Remove that student from the array using splice
- Send back the deleted student
- You need to write 6 to 8 lines of code

14. Niyonzima has an array of student objects. Each object has an id and a name. He wants to display each student's name inside a list item. The component is ready but the rendering is missing. **(3Marks)**

```
function StudentList() {
  const students = [
    { id: 1, name: "Amina" },
    { id: 2, name: "Bosco" },
    { id: 3, name: "Clarisse" }
  ];

  return (
    <ul>
    // ===== WRITE YOUR CODE HERE =====

    // ===== STOP WRITING HERE =====
    </ul>
  );
}

export default StudentList;
```

Expected Output: Three items showing each student name

What you need to do:

- Use the map() method to loop through the student array
- Return an for each student with a unique key prop
- You need to write 3 lines

15. Uwera stores her task list data like this:

(3Marks)

```
const task1 = "Buy milk";  
const task2 = "Study";  
const task3 = "Exercise";
```

Ndayisaba stores the same data like this:

```
const tasks = ["Buy milk", "Study", "Exercise"];
```

Analyze the two approaches. Identify 3 structural differences between the way Uwera and Ndayisaba organize this data. (1 mark each)

For each difference, name the aspect that is different and describe what each developer's code does in that aspect.

16. Mutoni built a Student API using Express. But she made these choices:

(5Marks)

- She hardcodes the port number as 3000 directly in her code
- She does not validate any input — if a user sends empty data, it still saves
- She uses one single route `app.get("/students")` for everything — getting all students, getting one student, and searching
- She does not use Nodemon, so she manually restarts the server after every code change
- She does not test her endpoints — she just opens the browser and hopes it works

Her instructor said: *"Your API works, but it is not built for a real project."*

Examine Mutoni's 5 choices. For each one, **compare** her method with the correct approach and **explain** why the correct method is better. (1 mark each)

For each of your 5 points:

- **Name** what Mutoni did wrong
- **Show** the correct way
- **Explain** why in one or two sentences

17. Bizimana and Uwamahoro are both building a **multi-page school portal** with pages for Home, Students, and Courses. Here is how each one does it: **(10marks)**

Area	Bizimana	Uwamahoro
Navigation	Creates 3 separate HTML files and uses <a> links	Uses React Router with <Route> and <Link> on one page
Active link	No way to highlight which page is active	Uses NavLink with active className
Student detail	Creates a separate page for each student manually	Uses dynamic routes like /students/:id with useParams
Page layout	Copies the header and footer into every page	Uses a Layout wrapper component with <Outlet />
Wrong URL	Shows a broken page when user types wrong URL	Uses a redirect or fallback route to handle unknown paths

Examine both approaches in these 5 areas. For each one, distinguish between the two methods by identifying what is different in how they handle that area. (2 marks each)

For each of your 5 points:

- Name the area
- Describe what Bizimana does and how it works
- Describe what Uwamahoro does and how it works

18. (10marks)

Uwamahoro builds her first Express server. She puts **all middleware, routes, database logic, validation, and error handling** in app.js. The file is 600 lines. Her mentor says:
"Your server works, but no team would accept this code."

(a) Explain why a 600-line single file is a problem for a team. Give **5 consequences**. **(5 marks)**

(b) Propose how to restructure the project into separate files. **Justify** 5 specific files/folders she should create. **(5 marks)**

19. (10marks)

Mugabo builds a form but when he clicks submit, the whole page reloads and the data is lost.

a) Judge why the page reloads and how to fix it. (5 marks)

b) Compare the two approaches. Which correctly handles form submission in React? (5 marks)

```
// Approach A: No preventDefault (page reloads)
function handleSubmit() {
  console.log("submitted");
}
<form onSubmit={handleSubmit}>
  <input type="text" />
  <button type="submit">Send</button>
</form>

// Approach B: With preventDefault (no reload)
function handleSubmit(e) {
  e.preventDefault();
  console.log("submitted");
}
<form onSubmit={handleSubmit}>
  <input type="text" />
  <button type="submit">Send</button>
</form>
```

20.

(10marks)

Mugisha loads all page components at the start even though the user may never visit some pages.

a) Judge if this is a problem. What does lazy loading solve? (5 marks)

b) Compare the two approaches. Which loads the About page only when needed? (5 marks)

```
// Approach A: Import all at start
import Home from "./Home";
import About from "./About";
import Contact from "./Contact";

// Approach B: Lazy load
import { lazy, Suspense } from "react";
const Home = lazy(() => import("./Home"));
const About = lazy(() => import("./About"));
const Contact = lazy(() => import("./Contact"));

<Suspense fallback={<p>Loading...</p>}>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
  </Routes>
</Suspense>
```

21.

(40marks)

Mugisha owns a bookstore in Kigali. Use Express and build an API to manage his books.

Start with this Express.js code to complete the requirements below.

```
const express = require('express');  
const app = express();  
const books = [];  
app.listen(3000);
```

- a) Set up an Express server and connect to MongoDB using Mongoose. **(8 marks)**
- b) Create a **Book** model with fields: title (String, required), author (String, required), price (Number, required). **(8 marks)**
- c) Create a POST route /api/books to add a new book and return the saved book. **(8 marks)**
- d) Create a GET route /api/books to return all books and a GET route /api/books/:id to return one book by ID. **(8 marks)**
- e) Create a PUT route /api/books/:id to update a book and a DELETE route /api/books/:id to delete a book. Return 404 if the book is not found. **(8 marks)**